

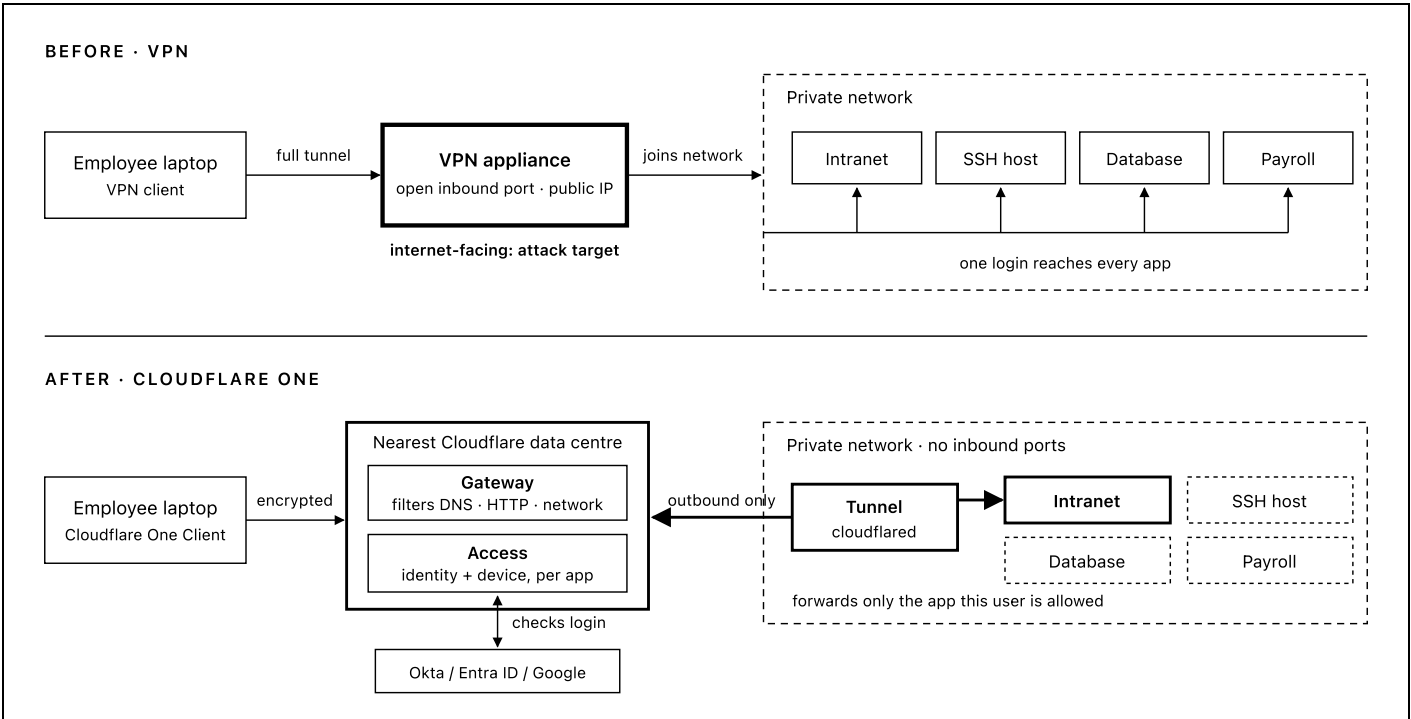
Under the Cloudflare Umbrella

One edge decides who may do what. Staff reach private apps through Cloudflare One instead of a VPN. Shoppers and AI agents reach commerce through a Worker that checks consent, capabilities and mandates against Xano before anything acts. Closed-fee middleware moves data between systems; it does not carry consent, erasure or authority with it.

Yoonsun Lee · Solutions architecture portfolio · September 2026 · [crm-sync.dev](#) · [github.com/persephonepunch](#) · [linkedin.com/in/yoonsunlee](#)

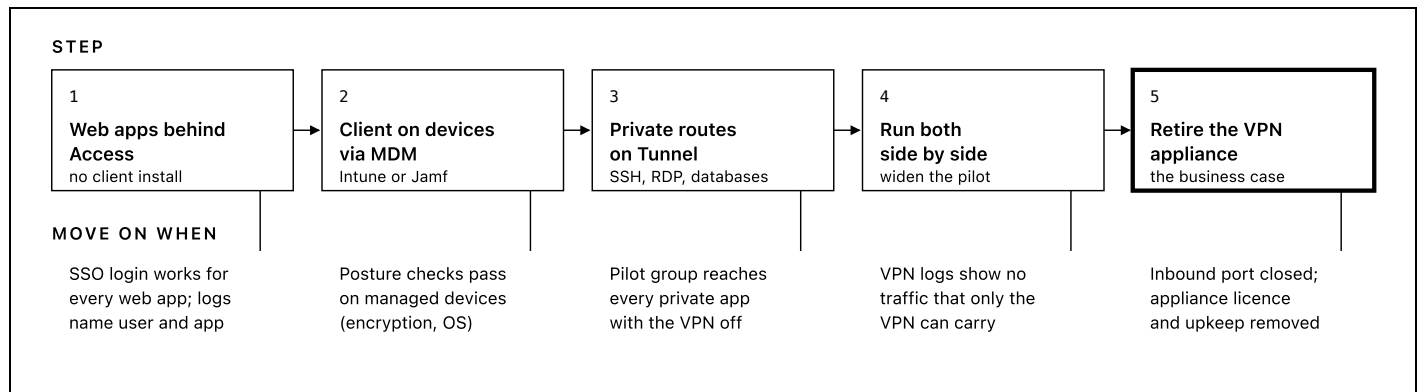
1 · [Staff: VPN to Zero Trust](#) · 2 · [Xano Auth and Google ADK](#) · 3 · [Middleware and CRM gaps](#) · 4 · [Permissions and rendering](#)

1 · Staff: what changes when the VPN goes



The attack surface and the reach both shrink. The VPN appliance and its open inbound port disappear; the tunnel connects out to Cloudflare, so nothing inside the network listens on the internet. After login, the user reaches only the app their policy names, and each request is checked again. Web-only apps need no client at all: Access alone sits in front of them.

How the migration runs



Each step has a condition to meet before the next one starts, and step 4 is the safety net: the VPN stays up until its own logs show nothing depends on it. Start with the web apps because they need no client and give users the quickest visible change.

Objections to replacing the VPN

"Our VPN works."

It also gives every connected user the whole network, and VPN appliances are among the most attacked devices on the internet. With per-app access, a stolen password opens one app.

"Our legacy apps won't work."

Anything that speaks TCP or UDP runs over the client and the tunnel, and web apps need no client at all. Pilot the hardest app first.

"It will be slower."

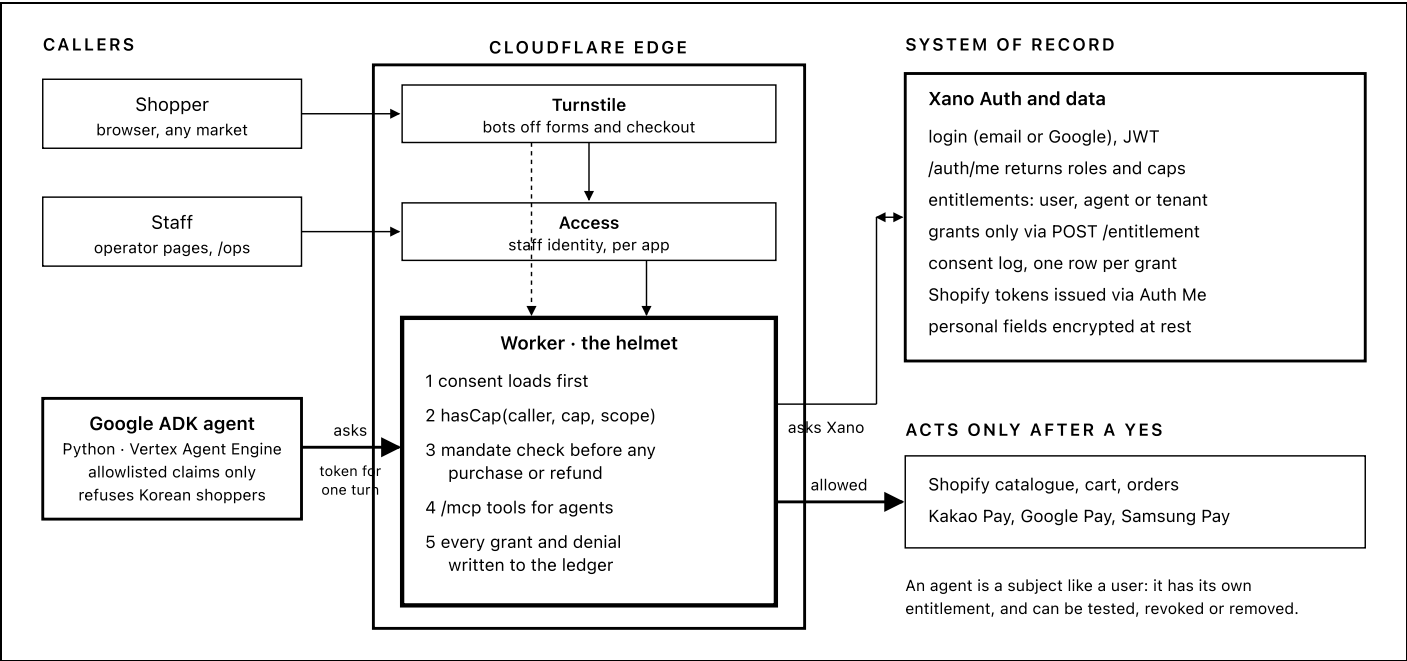
Users connect to the nearest Cloudflare data centre instead of hairpinning through one head office, which matters most for teams spread across a region like APAC.

"What do we show the auditor?"

Every allow and deny is logged per user and per app, which is the evidence SOC 2 access controls (CC6) ask for.

2 · Xano Auth and Google ADK under the umbrella

Every request, from a shopper, a staff member or an AI agent, passes the Cloudflare edge first. The Worker asks Xano who the caller is and what they may do, then acts or refuses. The agent never holds that decision.

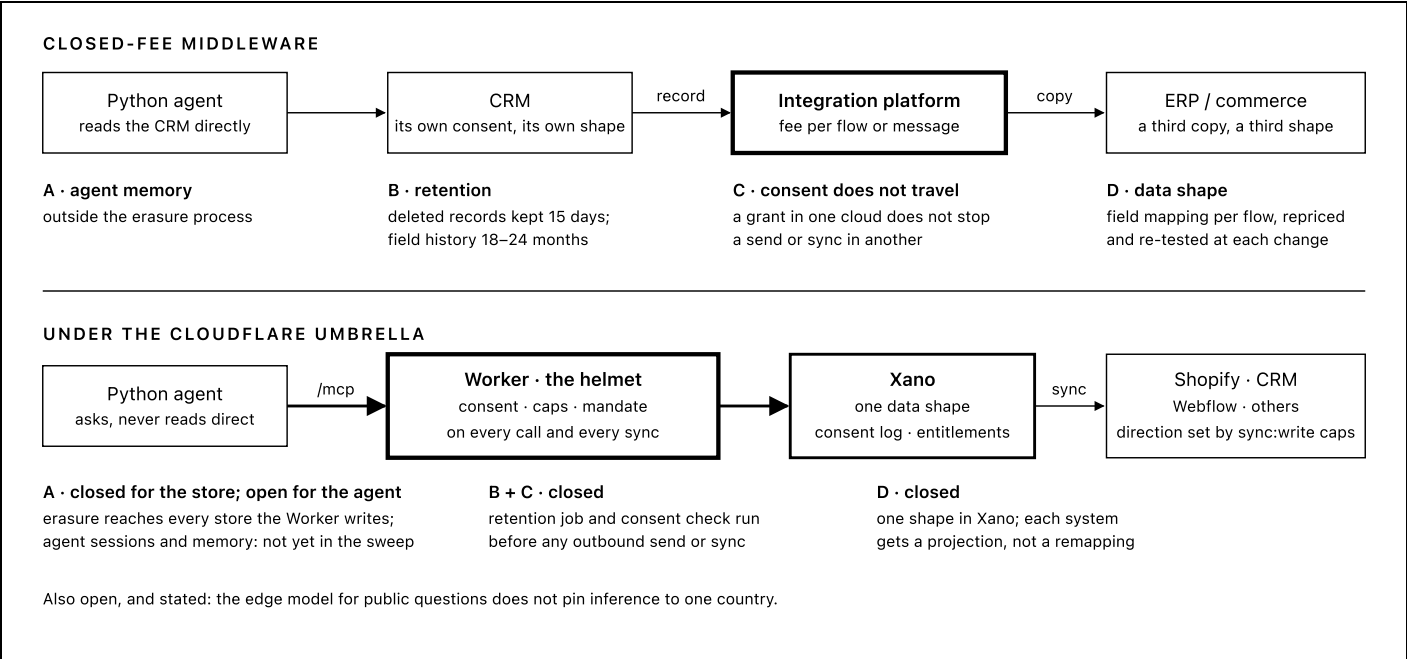


The agent is a shop assistant without the keys: it asks the Worker, the Worker decides with Xano, and the login token is lent for one step and taken back. Dashed line: a shopper's own requests also reach the Worker after Turnstile.

CAPABILITY	WHAT IT ALLOWS	WHO HOLDS IT
crm:agents:read	See registered agents and their activity	Content, audit, QA, security
crm:agents:test	Run an agent against the test gate	Release, QA, security
crm:agents:revoke	Switch an agent off	QA, security
crm:agents:manage	Create agents and change their permissions	Security only
crm:privacy:dsr:read / operate	Review privacy requests / run the erasure	Review: several teams · Operate: security and QA only
crm:keys:ceremony	Rotate a tenant's keys	Security (CISO)
mandate (AP2)	Let an agent buy within a cap, until a date	Granted per shopper, signed, revocable

3 · Closed-fee middleware and CRM: where the gaps are

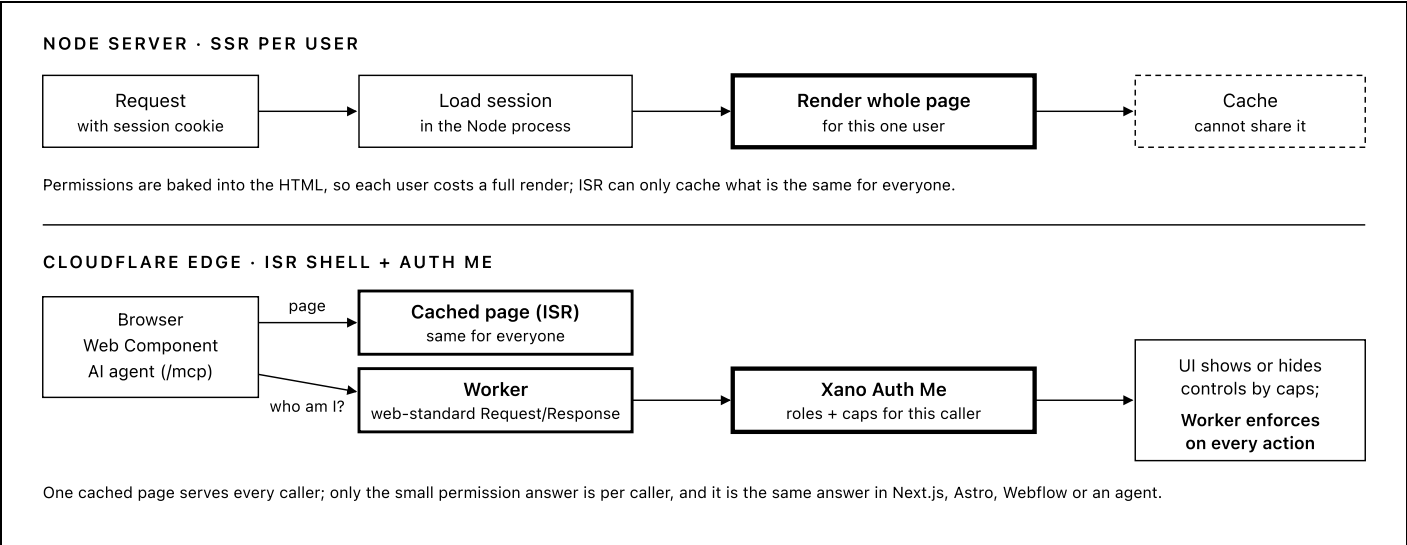
Integration platforms charge to move data between a CRM and commerce systems. They move the record; the consent, retention, erasure and authority around it stay each system's own problem. Under the umbrella, the Python agent and every sync pass one place that carries those rules.



4 · Permissions and rendering: why Auth Me changes SSR

On a Node server, personalising a page usually means rendering it per request (SSR), so nothing can be cached for everyone. With Xano Auth Me returning each caller's roles and capabilities, the page can stay one cached copy (ISR) at the edge, and only the permission answer is per caller. The same answer serves a browser, a Web Component in any framework, and an AI agent through /mcp.

The runtimes line up too. Custom code steps in Xano (Lambdas) run JavaScript or TypeScript on Deno, which uses the same web-standard Request and Response model as Cloudflare Workers. A typical AWS setup runs Node, with its own request objects and packaging. Code written against web standards moves between Xano and the edge with little change; code written for Node usually needs porting.



The rendering choice and the permission check come apart. Hiding a button is a convenience; the Worker's check on the action is the control, so a cached page never grants anything.

	NODE SERVER, SSR PER USER	EDGE ISR + XANO AUTH ME
Where permissions live	Baked into each rendered page	One Auth Me answer per caller: roles and capabilities
Caching	Personalised pages cannot be shared	The page is cached once and served to everyone
Who can use the answer	The one framework that rendered it	Browsers, Web Components in any framework, AI agents
Where the check is enforced	In the render, per framework	In the Worker, on every action, for every caller
Runtime model	A long-running Node process with its own request objects	Web-standard Request and Response in isolates, the same model Deno uses
Custom code in the backend	AWS Lambda functions, typically on Node	Xano Lambda steps: JavaScript or TypeScript on Deno; XanoScript for everything else

A VPN trusts the network, and middleware trusts the record. Under the Cloudflare umbrella nothing is trusted by default: the person, the device, the agent and the sync are each checked on every request, against one system of record.

Architecture from the CRM Sync estate: the Google ADK concierge is built and tested, not yet deployed. Product names as in Cloudflare's documentation, September 2026: the device software is the Cloudflare One Client (formerly WARP); site-to-site links run on Cloudflare WAN.